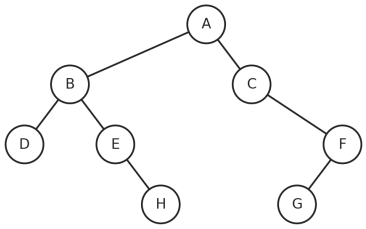
1. (4 Punkte) Beantworte die folgenden Fragen zum abgebildeten Baum.



   (a) (1 Punkt) Welcher Knoten ist die Wurzel?

   (b) (1 Punkt) Welche Knoten sind Blätter?

   (c) (1 Punkt) Wie heißen der Elternknoten von E und die Kinder von C?

   (d) (1 Punkt) Auf welcher Ebene liegt der Knoten G?

2. (3 Punkte) Die Wurzel eines binären Baums liegt auf Ebene 0.

   (a) (1 Punkt) Wie viele Knoten kann ein binärer Baum auf Ebene 3 höchstens haben?

   (b) (1 Punkt) Wie viele Knoten kann er auf Ebene k höchstens haben?

   (c) (1 Punkt) Wie viele Knoten hat ein Baum, bei dem alle Ebenen von 0 bis 4 vollständig besetzt sind?

3. (3 Punkte) Die Funktion `baumString` liefert für den Baum b die folgende Ausgabe. Zeichne den Baum.

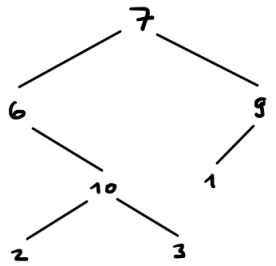
```
..6
.9
...1
..4
5
.2
..7
```

4. (3 Punkte) Zeichne den Baum, auf den die Variable w verweist.

```
x = Node(4, Node(7), Node(2))
y = Node(9, None, Node(1))
z = Node(3, x, y)
w = Node(8, z, None)
```

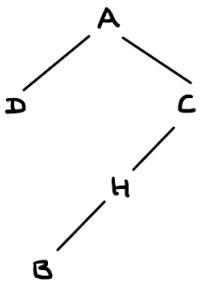
5. (5 Punkte) a. Schreibe eine Klasse `Node`, die einen Knoten eines binären Baums repräsentiert.

   b. Schreibe eine Funktion `baum`, die den abgebildeten Baum zurückgibt. Nutze dazu deine Node-Klasse. Verschachtele die Node-Aufrufe aus Lesbarkeitsgründen höchstens einmal und gib den temporären Teilbäumen sprechende Namen.



6. (5 Punkte) a. Schreibe eine Klasse `Node`, die einen Knoten eines binären Baums repräsentiert.

   b. Schreibe eine Funktion `baum`, die den abgebildeten Baum zurückgibt. Nutze dazu deine Node-Klasse. Verschachtele die Node-Aufrufe aus Lesbarkeitsgründen höchstens einmal und gib den temporären Teilbäumen sprechende Namen.



7. (3 Punkte) Gegeben ist das folgende Programm.

```
def f(node):
    if node is None:
        return 0
    return node.val + f(node.left)

baum = Node(2, Node(5, Node(1), Node(9)), Node(4))
print(f(baum))
```

   (a) (2 Punkte) Welche Ausgabe erscheint auf der Konsole?

   (b) (1 Punkt) Beschreibe in einem Satz, was die Funktion f berechnet.

8. (3 Punkte) Führe die Funktion von Hand aus. Welche Ausgabe erscheint auf der Konsole?

```
def g(node):
    if node is None:
        return
    g(node.left)
    g(node.right)
    print(node.val, end=' ')

baum = Node(1, Node(2, None, Node(3)), Node(4, Node(5), Node(6)))
g(baum)
```

9. (3 Punkte) Die Funktion `summe(node)` gibt die Summe aller Werte im Baum mit der Wurzel node zurück. Für einen leeren Baum ist das Ergebnis 0. Implementiere die Funktion rekursiv.

10. (3 Punkte) Die Funktion `enthaelt(node, x)` gibt True zurück, wenn der Wert x im Baum mit der Wurzel node vorkommt, sonst False. Implementiere die Funktion rekursiv.

11. (4 Punkte) Die Funktion `blaetter(node)` gibt die Anzahl der Blätter im Baum mit der Wurzel node zurück. Implementiere die Funktion rekursiv.

12. (3 Punkte) Die Funktion `anzahl` soll die Anzahl der Knoten eines Baums zurückgeben.

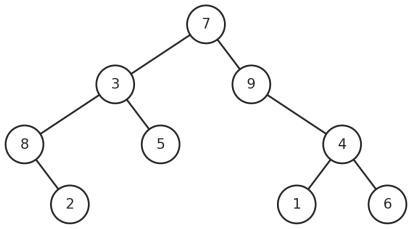
```
def anzahl(node):
    return 1 + anzahl(node.left) + anzahl(node.right)

print(anzahl(Node(5)))
```

(a) (2 Punkte) Was passiert beim Aufruf? Begründe.

(b) (1 Punkt) Korrigiere die Funktion.

13. (4 Punkte) Gegeben ist der abgebildete Baum.



(a) (2 Punkte) In welcher Reihenfolge besucht die Tiefensuche die Knoten? (erst der Knoten, dann der linke, dann der rechte Teilbaum)

(b) (2 Punkte) In welcher Reihenfolge besucht die Breitensuche die Knoten?

14. (4 Punkte) Für den Baum aus der vorigen Aufgabe wird eine Tiefensuche mit einem Keller durchgeführt. Notiere zeilenweise die Ausgabe und den Inhalt des Kellers, nachdem das rechte Kind (falls vorhanden) nach der Ausgabe in den Keller getan wurde. Notiere den Keller horizontal mit dem top-Element auf der linken Seite.

15. (4 Punkte) Für denselben Baum wird eine Breitensuche durchgeführt. Notiere zeilenweise die Ausgabe und den Inhalt der Schlange, nachdem die Kinder (falls vorhanden) nach der Ausgabe in die Schlange eingefügt wurden. Notiere die Schlange horizontal mit dem Kopf der Schlange auf der linken Seite.

16. (3 Punkte) Vergleiche Tiefensuche und Breitensuche.
    (a) (1 Punkt) Welche Datenstruktur nutzt die Tiefensuche, um sich die noch offenen Teilbäume zu merken, welche die Breitensuche?
    (b) (1 Punkt) Warum eignet sich für die Breitensuche keine Datenstruktur, die nach dem LIFO-Prinzip arbeitet?
    (c) (1 Punkt) Ein gesuchter Wert liegt nah an der Wurzel in einem sehr großen Baum. Welches Verfahren findet ihn in der Regel schneller? Begründe.

17. (4 Punkte) Erkläre in ein bis zwei Sätzen.
    (a) (1 Punkt) Was ist ein binärer Baum?
    (b) (1 Punkt) Was ist ein Blatt?
    (c) (1 Punkt) Was bedeutet es, einen Baum zu traversieren?
    (d) (1 Punkt) Warum ist der Baum eine rekursive Datenstruktur?